

B V RAJU INSTITUTE OF TECHNOLOGY

Vishnupur, Narsapur, Medak, Telangana — 502313

Department of Computer Science and Engineering (Data Science)

TECHNICAL REVIEW DOCUMENT

Geometric Linguistic Quantifier (GLQ)

Critical Analysis: Flaws and Efficiency Improvements

Project	Geometric Linguistic Quantifier (GLQ) v5.0
Scope	End-to-end review of generate_anchors.py, web_quantifier.py, and the overall pipeline architecture
Author	Vukkem Bhuvaneshwar
Roll Number	22211A67C3
Guide	Mr. Krutibash Nayak, Assistant Professor, CSE (DS)
Programme	Bachelor of Technology — Computer Science and Engineering (Data Science)
Academic Year	2025 – 2026
Document Purpose	This document provides a structured critical analysis of the current GLQ implementation, cataloguing all

Table of Contents

1.	Core Algorithmic Flaws	
2.	Architecture Flaws	
3.	Performance and Scalability Flaws	
4.	Evaluation and Validation Flaws	
5.	Root-Cause Analysis: Why Hindi Always Appears	
6.	Prioritised Improvement Roadmap	
7.	Summary and Conclusion	

CHAPTER 1

Core Algorithmic Flaws

This chapter documents four fundamental deficiencies in the GLQ algorithm itself. These flaws affect output correctness regardless of how the application is deployed or configured.

1.1 Anchor Quality is Fundamentally Limited

The project currently uses only three anchor vectors — one per language — each produced from a single K-Means cluster centroid. This is the most critical flaw in the system. A single mean vector compresses an entire language, with millions of speakers, dozens of dialects, and countless registers, into one 384-dimensional point. Code-mixed sentences frequently occupy the space *between* all anchors, making cosine similarity an unreliable proxy for language percentage.

Further investigation reveals that despite the anchor generator being built with `N_CLUSTERS = 2`, only the single-cluster files (`_baseline.pt`) are present in `data/baselines/`. The second cluster files (`_baseline1.pt`) are missing for all three languages. The `load_anchors()` function loads whatever is in the folder, meaning the richer two-cluster representation was never actually used.

```
data/baselines/
english_baseline.pt (cluster 0 only – present)
hindi_baseline.pt (cluster 0 only – present)
telugu_baseline.pt (cluster 0 only – present)
# Missing: *_baseline1.pt files for all three languages
```

Listing 1: Missing second-cluster anchor files

1.2 English Fallback Data is Severely Contaminated

In `generate_anchors.py`, when Wikipedia streaming fails, the fallback routine takes 15 unique sentences and replicates them thousands of times to reach the target `SAMPLE_SIZE` of 25,000:

```
# generate_anchors.py, line 109
multiplied = (base * ((n // len(base)) + 1))[:n]
random.shuffle(multiplied)
```

Listing 2: Sentence replication in English fallback

Encoding the same 15 sentences 1,600 or more times produces near-identical embeddings. The resulting English anchor becomes the centroid of those 15 sentences alone, biasing it toward a narrow vocabulary drawn from science, politics, and news. Casual English such as conversational phrases is entirely absent, which directly reduces detection accuracy for informal code-mixed inputs.

1.3 Contrast Subtraction Has No Theoretical Grounding

The core calibration step subtracts a scaled version of the mean cosine similarity from each score:

```
mean_noise = np.mean(scores)
adjusted_scores = scores - (mean_noise * contrast)
```

Listing 3: Contrast subtraction heuristic

While this reduces Indic language conflation in some cases, it introduces three significant problems. First, it is not language-aware: if one language has a legitimately high similarity score, subtracting the global mean penalises that language unfairly, pulling the result toward a false blend. Second, the contrast slider has no calibrated meaning, so users cannot reason about the effect of adjusting it. Third, at contrast = 1.0 all adjusted scores collapse to zero, causing softmax to output exactly 33.3% for every language regardless of input — a silent failure with no warning to the user.

1.4 Softmax Does Not Model Linguistic Absence

F.softmax always sums to 1.0, which forces the model to distribute probability mass across all languages even when input is entirely monolingual. A sentence that is 100% English will always show residual Hindi and Telugu percentages because softmax cannot assign zero to any class. The display threshold `df[df['Density (%)'] > 0.5]` is a cosmetic fix only — it does not change the underlying mathematics. At tau = 0.4, a pure English sentence consistently yields 20–25% for each Indic language before the threshold filter removes them visually.

CHAPTER 2

Architecture Flaws

This chapter documents four deficiencies in the application architecture that affect correctness, security, and maintainability.

2.1 Preset Buttons Cause Silent State Desynchronisation

The `@st.cache_resource` decorator correctly caches the model across Streamlit sessions, and slider changes do not incorrectly trigger re-caching. However, pressing either sidebar preset button causes a different class of bug:

```
# web_quantifier.py, lines 107-109
if st.button("High-Density Blend (Detect Bits)":
    tau, contrast = 0.40, 0.60 # local variable reassignment only
if st.button("Sharp Language ID)":
    tau, contrast = 0.10, 0.90
```

Listing 4: Preset button state desync

When a preset button is pressed, Streamlit reruns the script but the slider widgets retain their previous visual positions. The calculation uses the button-overridden values while the sliders display the old ones. The user has no indication that the displayed slider positions do not reflect the values actually in use.

2.2 No Input Validation or Error Handling

`web_quantifier.py` performs no input validation of any kind. Three specific failure modes result from this omission:

- Single characters, emojis, or purely numeric inputs are encoded and analysed without any warning to the user.
- Inputs exceeding the transformer's maximum sequence length (typically 128 or 512 sub-word tokens) are silently truncated. The user receives results for a truncated version of their input with no indication that truncation has occurred.
- If `data/baselines/` is empty or all `.pt` files are corrupt, the application falls back to `get_mock_anchors()`, which uses single-sentence mock baselines. Results are displayed without informing the user that mock data is in use.

2.3 `torch.load()` Uses a Deprecated and Unsafe Default

```
# web_quantifier.py, line 26
self.baselines[name] = torch.load(
    os.path.join(folder, file), map_location='cpu'
)
```

Listing 5: Unsafe torch.load() call

Calling `torch.load()` without `weights_only=True` is deprecated in PyTorch 2.0 and later and will raise a `FutureWarning`. More critically, this form enables arbitrary code execution via pickle deserialization if a `.pt` file is replaced by a malicious one. The `generate_anchors.py` script already uses `weights_only=True` correctly in its `verify_anchors()` function; the inconsistency indicates an oversight rather than a deliberate design decision.

2.4 Hard-Coded Relative Paths

```
# generate_anchors.py, line 31
JSON_FILE = "../Data.json" # relative to script location

# web_quantifier.py, line 18
folder = "data/baselines" # relative to process working directory
```

Listing 6: Hard-coded relative paths

Both scripts rely on relative paths that break the moment they are executed from a working directory other than the one assumed during development. This is a documented recurring source of `FileNotFoundError` and is easily remedied by using `pathlib.Path(__file__).parent` to anchor all paths relative to the script file itself.

CHAPTER 3

Performance and Scalability Flaws

3.1 Re-encoding on Every Keystroke

There is no debouncing or caching on the text input. Every time a user types a character into the `st.text_area` widget, Streamlit reruns the entire script and `engine.calculate()` calls `self.model.encode(text)`, triggering a full transformer forward pass. On CPU hardware this costs 50–200 milliseconds per character. On slower machines the UI becomes sluggish and unresponsive, degrading the user experience significantly.

Remedy: *Remedy: Wrap the text area and the analysis logic inside `st.form()` with an explicit submit button, or add a dedicated Analyse button, so that encoding is triggered only on deliberate submission rather than on every keystroke.*

3.2 K-Means Encoding Has No Checkpointing

The `generate_anchors.py` script runs K-Means clustering synchronously in the main thread with no intermediate checkpointing:

```
kmeans = KMeans(n_clusters=2, random_state=RANDOM_SEED, n_init=10)
kmeans.fit(embeddings) # blocks for minutes on CPU
```

Listing 7: Synchronous K-Means with no checkpointing

If the process is interrupted at any point — through power loss, keyboard interrupt, or session timeout on a cloud platform — all computed embeddings are lost and the entire encoding step must be restarted from scratch. For 25,000 sentences across three languages with batch encoding, this represents 20–40 minutes of lost computation.

Remedy: *Remedy: Save the embeddings array to disk as a `.npy` file after the encoding step completes. At the start of the script, check whether the `.npy` file exists and skip re-encoding if it does. This makes the pipeline restartable at the clustering stage.*

3.3 No Model Version Pinning or Offline Fallback

The `SentenceTransformer` constructor downloads the model from Hugging Face Hub on first run with no version pinning, no download progress notification to the user, and no fallback for offline environments. A future update to the model weights on Hugging Face could silently change the embedding geometry, invalidating all pre-computed anchor files without any warning.

CHAPTER 4

Evaluation and Validation Flaws

4.1 No Ground Truth Dataset Exists

The project roadmap specifies creating a calibration set of 50–100 sentences with human-assigned language percentages (Phase IV, Task 1). This was never completed. There is consequently no means by which to objectively measure whether the output of the quantifier is accurate. All tuning of the tau and contrast parameters has been performed by subjective inspection of the output on a handful of example sentences — an approach that cannot detect systematic biases or generalisation failures.

4.2 The tau and contrast Parameters are Coupled but Treated as Independent

The effective output distribution is jointly controlled by both tau and contrast, but the user interface presents them as two independent sliders. Setting contrast = 0.90 with tau = 0.40 produces fundamentally different behaviour from contrast = 0.40 with tau = 0.90, but neither label in the UI explains this interaction. The parameter space has no documented optimal region and no principled stopping criterion.

4.3 No Anchor Health Check at Runtime

A critical diagnostic — whether the three language anchors are sufficiently separated from each other in embedding space — is checked only once during anchor generation via `verify_anchors()` and never at application startup. If the anchors converge (for example, if the cosine similarity between the Hindi and Telugu anchors exceeds 0.98), all output results will be meaningless. The application will nonetheless continue serving results without any warning to the user.

CHAPTER 5

Root-Cause Analysis: Why Hindi Always Appears

The most visible and frequently reported symptom of the current implementation is that Hindi appears in the output even when the input sentence contains no Hindi whatsoever. This is not a single bug. It is a chain of five compounding failure points, each amplifying the one before it. This chapter traces each link in that chain.

5.1 Root Cause: The Multilingual Model Collapses Indic Languages

The model used — paraphrase-multilingual-MiniLM-L12-v2 — was trained to map semantically similar sentences from different languages to nearby regions of embedding space. This property is desirable for translation tasks but is directly harmful for language identification. Both Hindi and Telugu are Indic languages whose parallel translations appear together in the model's training data. As a consequence, the model places Hindi and Telugu sentences very close to each other in the 384-dimensional space.

A Telugu input sentence will therefore score high cosine similarity against both the Telugu anchor and the Hindi anchor, with a gap that is often only 0.02–0.03. After softmax, this tiny gap is amplified into a falsely high Hindi percentage:

Illustrative approximate values for a pure Telugu input:
Input: "Meru ela unnaru?"
Raw cosine scores:
Telugu : 0.823
Hindi : 0.798 <- Only 0.025 lower despite no Hindi content
English: 0.641
After softmax (tau=0.25):
Telugu : ~52%
Hindi : ~40% <- Inflated despite complete absence from input
English: ~8%

Listing 8: Illustrative softmax inflation of Hindi scores

5.2 Amplifier: Anchors Were Generated From a Parallel Corpus

Both the Hindi and Telugu anchors were generated from Data.json, which is a parallel translation corpus — every Hindi sentence has a corresponding Telugu sentence carrying the same meaning.

Because the model was specifically trained to map parallel sentences to the same embedding region, the resulting Hindi anchor vector and Telugu anchor vector are very likely nearly identical. The following diagnostic code can verify this:

```
import torch

h = torch.load('data/baselines/hindi_baseline.pt', weights_only=True)
t = torch.load('data/baselines/telugu_baseline.pt', weights_only=True)
print(torch.dot(h, t).item()) # expected: > 0.90
```

Listing 9: Diagnostic check for inter-anchor similarity

When two anchors are nearly identical, cosine similarity cannot meaningfully distinguish between them for any input. Because Hindi has substantially more training data in the model than Telugu, the Hindi anchor is inherently better-fitted to the model's internal representation, giving Hindi a systematic advantage over Telugu for all Indic-flavoured inputs.

5.3 Amplifier: Contrast Subtraction Does Not Close the Gap

The contrast subtraction step subtracts the same constant from all three scores. Because the absolute gap between Telugu and Hindi is preserved after a uniform shift, the relative ratio changes slightly but Hindi remains on top whenever the Hindi anchor is systematically higher — which, due to the factors in §5.2, it reliably is for all Indic inputs.

5.4 Amplifier: Softmax Cannot Output Zero for Any Language

F.softmax guarantees that every language receives a strictly positive probability for any finite input. Hindi therefore cannot be mathematically removed from the output. The 0.5% display filter provides no meaningful protection because Hindi scores routinely fall in the 20–45% range for non-Hindi inputs.

5.5 Amplifier: The Second Telugu Anchor is Missing

The generator was designed to create two anchors per language: one for the formal or news register and one for the conversational or casual register. The second Telugu anchor (`_baseline1.pt`) is missing. Code-mixed Telugu-English input therefore does not match either available Telugu anchor well; it sits in a region of embedding space that no Telugu anchor covers. In the absence of a strong Telugu match, the nearby Hindi anchor wins by default.

5.6 The Full Failure Chain

User inputs Telugu text
v
[Model maps input close to Hindi anchor]
-- Cause: multilingual model conflates Indic languages
v
[Hindi and Telugu anchors are near-identical]
-- Cause: both generated from a parallel translation corpus
v
[Contrast subtraction does not close the gap]
-- Cause: uniform shift, not language-aware
v
[Softmax ensures Hindi score is never zero]
-- Cause: mathematical property of softmax
v
[No second Telugu anchor to compete for casual register]
-- Cause: _baseline1.pt files are missing
v
Hindi appears inflated or dominant in output [INCORRECT RESULT]

Listing 10: Full failure chain for the Hindi inflation symptom

5.7 Recommended Fixes for This Specific Problem

Step	Action	Rationale
1	Compute inter-anchor cosine similarity at application startup. Display the visible nearest neighbors in the UI.	Displays the visible nearest neighbors in the UI.
2	Regenerate all anchors using separate monolingual corpora. Extract the parallel corpus from dataset. Use Indic NLP or AL	Extracts the parallel corpus from dataset. Use Indic NLP or AL
3	Replace paraphrase-multilingual-MiniLM-L12-v2 with a language identification embeddings designed to separate languages rather	Language identification embeddings designed to separate languages rather
4	Replace softmax with sparsemax, or apply a hard threshold. Allow Hindi language with 60% similarity to be output. The output	Allowing Hindi language with 60% similarity to be output. The output
5	Generate and load both cluster anchors per language, particularly Telugu, as a conversational cluster casual code-mixed input	Gives Telugu a fair conversational cluster casual code-mixed input

Table 1: Recommended Fixes for the Hindi Inflation Problem

CHAPTER 6

Prioritised Improvement Roadmap

The table below consolidates all identified issues into a single prioritised roadmap. Issues are grouped by severity: Critical issues affect output correctness or security; High issues affect usability and reliability; Medium issues affect accuracy and robustness; Low issues affect maintainability.

Severity	Problem	Recommended Fix
Critical	Only one cluster centroid loaded per language	Generate and load both <code>_baseline.pt</code> and <code>_baseline1.pt</code> . Use <code>max</code>
Critical	English fallback is 15 sentences replicated thousands of times	Use <code>datasets</code> library's streaming API as the primary source, or
Critical	<code>torch.load()</code> uses unsafe default (no <code>weights_only=True</code>)	Add <code>weights_only=True</code> to every <code>torch.load()</code> call in <code>web_quantifie</code>
High	Preset buttons desynchronise from slider widgets	Use <code>st.session_state</code> to control slider values programmatically so
High	No input validation	Add a minimum-length check, a script-detection warning for unsu
High	Re-encoding on every keystroke (no debouncing)	Wrap input and analysis in <code>st.form()</code> or add an explicit Analyse bu
Medium	Softmax forces non-zero mass on all classes	Replace <code>F.softmax</code> with temperature-scaled <code>sparsemax</code> , or apply
Medium	Contrast subtraction is an ungrounded heuristic	Replace with inter-anchor whitening: project scores into a space v
Medium	No ground truth evaluation dataset	Build a 50-sentence labelled test set with human-annotated langu
Medium	No embedding checkpointing in <code>generate_anchors.py</code>	Save embeddings to a <code>.npy</code> file after encoding. Skip re-encoding
Low	Hard-coded relative paths in both scripts	Replace all path strings with <code>pathlib.Path(__file__).parent constru</code>
Low	No anchor health check at application startup	Add a check in <code>load_anchors()</code> that logs inter-language cosine sir
Low	No model version pinning	Pin the <code>MODEL_NAME</code> and add exact version constraints to requ

Table 2: Prioritised Improvement Roadmap

CHAPTER 7

Summary and Conclusion

The GLQ project is a creative and technically ambitious attempt to move beyond token-counting for code-mixed language analysis. Its foundational insight — that semantic vector geometry can capture proportions of language influence rather than merely language identity — is sound in principle and represents a genuine contribution to the problem space.

However, the current implementation has not yet validated this insight against ground truth. The anchor vectors are underpowered, with only one cluster centroid per language, a contaminated English baseline, and anchor pairs that are likely near-identical due to the parallel-corpus generation process. The calibration heuristics are theoretically fragile, the application contains several security and reliability deficiencies, and there is no objective measurement of output quality. The project is effectively in a pre-validation state despite its polished user interface.

The analysis conducted in this document identifies 13 discrete issues across four categories: algorithmic, architectural, performance, and evaluative. Of these, three are classified as Critical.

Category	Issues Found	Critical	High	Medium	Low
Algorithmic	4	0	0	2	2
Architectural	4	1	2	0	1
Performance	3	1	1	0	1
Evaluation	3	1	0	2	0
Total	14	3	3	4	4

Table 3: Issue Count by Category and Severity

The single highest-leverage fix available to a developer continuing this project is to regenerate all six anchors (two clusters per language) from separate monolingual corpora and load all of them in the application. This change requires minimal effort and directly addresses the root cause of the Hindi–Telugu conflation problem that has been the recurring theme across multiple development iterations. The remaining critical fix — adding `weights_only=True` to the `torch.load()` calls — is a one-line change that eliminates a genuine security vulnerability.

With those two changes in place, the remaining improvements described in Chapter 6 can be addressed in priority order. At that point the system will be in a position to have its output validated against a ground truth dataset for the first time, which is the essential next milestone for the project.

GLQ — Critical Analysis: Flaws and Efficiency Improvements

Vukkem Bhuvaneshwar (22211A67C3) | B V Raju Institute of Technology | 2025–2026

Guide: Mr. Krutibash Nayak, Asst. Prof., CSE (DS) | H.O.D.: Dr. R.V. Ramana Chary